

Kaarten op schaal

QGIS gebruiken om honderden kaarten te genereren

Github repo:

`github.com/paulbradshaw/QGIS_param`

Na 1 uur weet je:

- Hoe je de **Python-functionaliteit van QGIS** gebruikt
- Hoe je **parameterisatie** gebruikt om een kaart te maken voor elk gebied/partner/stad
- Tips voor het gebruik van **AI** bij het coderen

Scenario:

De BBC Shared Data Unit werkt aan een artikel over waterkeringen. Dit zal naar redacties door heel het Verenigd Koninkrijk gaan. We willen voor de redactie van elke stad een kaartafbeelding genereren die de staat van de waterkeringen in hun werkgebied toont.

**Hoe maak ik een
kaart?**

Doe het nu:

Zoek het shape-bestand

Importeer in QGIS

Volg de handleiding in de repo

1. Ga naar [AIMS Spatial Flood Defences \(inc. standardised attributes\)](#) en download het bestand met de naam `AIMS_Spatial_Flood_Defences_inc_standardised_attributes.shp.zip`
2. Pak dat bestand uit
3. Open QGIS en maak een nieuw project (*Project > New*). Sla het op in dezelfde map.
4. Selecteer *Layer > Add Layer > Add vector layer*.
5. Klik in het venster dat verschijnt op ... naast het onderdeel Bron (waar staat *Vector Dataset(s)*) en zoek het bestand in de gedownloade map dat eindigt op **.shp**. Klik op **Add**. Dit zou op de achtergrond achter het venster moeten verschijnen. Klik op **OK** en dan op **Close** om terug te keren.
6. In het *Browser*-gebied linksboven in het scherm, scroll omlaag naar het onderdeel XYZ Tiles. Dubbelklik op OpenStreetMap om die basislaag aan uw project toe te voegen (klik op OK om de wijziging naar hetzelfde CRS goed te keuren). Dit zou nu in uw Lagen-gebied moeten verschijnen.
7. Klik en sleep de OSM-laag om deze onder de waterkeringenlaag te plaatsen.

Pas de weergave aan

1. Klik met rechts op de waterkeringenlaag en selecteer **Properties...**
2. Selecteer **Symbology** om de kleur, dekking, breedte, stijl enz. van de lijnen/vormen te wijzigen

**Hoe exporteer ik
een afbeelding?**

Doe het nu:

Zoom in op een gebied

Project > New Print

Layout...

Maak een opmaak (**dit gaan we automatiseren**)

1. Zoom in op het deel van de kaart waarvan je een afbeelding wilt maken (bijv. ingezoomd op een stad)
2. Ga naar **Project > New Print Layout...** Geef het een naam (bijv. stadzoom) en klik op **OK**
3. Het opmaakvenster werd leeg geopend. Klik op de knop Kaart toevoegen aan de linkerkant (paginapictogram met een groen plusteken)
4. Klik en sleep om het gebied te tekenen waar je de kaart wilt weergeven.
5. Ga naar het tabblad **Item Properties** aan de rechterkant en scroll omlaag naar **Position and Size**. Pas dit aan naar de gewenste afmetingen (verander de maat naar pixels als het voor het web is).
6. Schakel over naar het tabblad **Layout** ernaast, en scroll omlaag naar de knop **Resize** opmaak. Klik erop. De afbeelding zou nu het hele canvas moeten beslaan (je kunt inzoomen met de zoomknoppen in de linkerbovenhoek)
7. Als je tevreden bent met de afbeelding ga je naar **Layout > Export as Image**
8. Sluit het venster Afdrukopmaak en keer terug naar het QGIS-project



**Hoe doe ik dat
meerdere keren?**

Als je iets meer dan één keer moet doen...

Plugins > Python Console

Schrijf/open Python in de Console **onder de kaart**.

*Klik op de knop **Show Editor** om .py-bestanden te openen/bewerken*

PyQGIS library ([documentatie](#))

Doe het nu:

**Download het .py-
bestand**

Open het in de Console

Standardise

Python Console

```
1 # Python Console
2 # Use iface to access QGIS API interface or type '?' for more info
3 # Security warning: typing commands from an untrusted source can harm your computer
4
```

>>>



Untitled-0 X

1

Coordinate

357074,195749



Scale

1:152681



Magnifier

100%

Rotation

0.0 °



Render



EPSG:27700



(Stappen in deel 3 van de handleiding in de repo)

1. Ga naar het [raw-bestand](#) op GitHub. Klik op Bestand > Opslaan als... in je browser en sla het bestand op in je projectmap
2. Verwijder het commentaarteken van deze regel en verander het pad naar je projectmap

```
# out_prefix = "/Users/paul/Downloads/testqgis/images/"
```

(Om het pad naar je map op een Mac te vinden, klik je er met rechts op en selecteer je Info. Het venster dat verschijnt toont het pad onder 'Locatie:')

3. Open het bestand in de QGIS Python Console en voer daarna de code uit


```
# Laad tools waarmee we QGIS-kaarten en -indelingen vanuit dit script kunnen beheren
```

```
from qgis.PyQt.QtCore import QRectF
```

```
from qgis.core import (
```

```
    QgsProject, QgsCoordinateReferenceSystem, QgsPrintLayout, QgsLayoutItemMap,
```

```
    QgsLayoutPoint, QgsLayoutSize, QgsUnitTypes, QgsLayoutItemLabel,
```

```
    QgsLayoutExporter, QgsRectangle, QgsCoordinateTransform, QgsPointXY
```

```
)
```

```
# Laad math voor getallen, re (regex) voor tekst, os voor bestandsnavigatie
```

```
import math, re
```

```
import os
```

Hier wordt de uitvoermap gedefinieerd

Gebruik de os-bibliotheek om een map op je computer te maken voor de geëxporteerde kaartafbeeldingen.

Deze wordt geplaatst in de map Downloads, in een submap genaamd "qgis_images".

Als de map al bestaat, doet dit niets – er wordt niets overschreven.

```
out_dir = os.path.join(os.path.expanduser("~"), "Downloads", "qgis_images")
```

```
os.makedirs(out_dir, exist_ok=True)
```

```
out_prefix = out_dir + "/"
```

Om de hierboven ingestelde map te overschrijven, verwijder het commentaarteken van de onderstaande regel en pas deze aan

```
# out_prefix = "/Users/paul/Downloads/testqgis/images/"
```

```
#Stel het coördinatenreferentiesysteem (CRS) in - EPSG:4326 wordt gebruikt voor GPS  
#maar 27700 wordt gebruikt voor Britse waterkeringen, dus we gebruiken dit voor meer nauwkeurigheid
```

```
QgsProject.instance().setCrs(QgsCoordinateReferenceSystem("EPSG:27700"))
```

```
# Het huidige geopende QGIS-project ophalen en opslaan in een variabele genaamd project
```

```
project = QgsProject.instance()
```

```
#EPSG:27700 gebruiken betekent dat we lat/long moeten omzetten naar BNG-coördinaten  
# eerst het bronsysteem (EPSG:4326) en doelsysteem (EPSG:27700) instellen
```

```
source_crs = QgsCoordinateReferenceSystem("EPSG:4326")
```

```
dest_crs = QgsCoordinateReferenceSystem("EPSG:27700")
```

```
#dan een transformator aanmaken die die twee gebruikt
```

```
transformer = QgsCoordinateTransform(  
    source_crs,  
    dest_crs,  
    project  
)
```

#maak een nieuw afdruklay-out in dat project en sla dit op in een variabele genaamd layout

layout = QgsPrintLayout(project)

#vul het in met standaardinstellingen

layout.initializeDefaults()

#geef het een naam in QGIS

layout.setName("centres_layout") # Geef de lay-out een naam

Voeg een kaartvenster toe aan de lay-out – dit is de rechthoek die de kaart daadwerkelijk weergeeft.

```
map_item = QgsLayoutItemMap(layout)
```

```
map_item.setFrameEnabled(True) # Teken een rand rondom het kaartvenster
```

Positioneer het kaartvenster 20 mm van links en 20 mm van de bovenkant van de pagina.

```
map_item.attemptMove(QgsLayoutPoint(20, 20, QgsUnitTypes.LayoutMillimeters))
```

Stel het kaartvenster in op 257 mm breed en 170 mm hoog (ongeveer A4-liggend min marges).

```
map_item.attemptResize(QgsLayoutSize(257, 170, QgsUnitTypes.LayoutMillimeters))
```

```
layout.addLayoutItem(map_item) # Voeg het kaartvenster toe aan de lay-outpagina
```

Voeg een tekstlabel toe aan de lay-out, dat de naam en coördinaten van elke locatie weergeeft.

```
label = QgsLayoutItemLabel(layout)
```

```
layout.addLayoutItem(label)
```

Positioneer het label bovenaan de pagina (20 mm van links, 5 mm van boven).

```
label.attemptMove(QgsLayoutPoint(20, 5, QgsUnitTypes.LayoutMillimeters))
```

def maakt een functie (een recept) die we later kunnen gebruiken. Deze functie berekent een rechthoekig kaartgebied gecentreerd op een opgegeven punt. Er zijn vier ingrediënten nodig: x en y van het middelpunt; en de halve breedte/hoogte (met een standaardwaarde voor het laatste item als dit niet wordt opgegeven).

```
def rect_around(x, y, half_width, half_height=None):
```

```
    # Als er geen hoogte is opgegeven, gebruik dan dezelfde waarde als de breedte
```

```
    if half_height is None:
```

```
        half_height = half_width
```

```
    # Wanneer gebruikt, geeft dit een rechthoek terug die kan worden gebruikt om in te stellen waar de kaart op is gericht.
```

```
    return QgsRectangle(
```

```
        x - half_width,
```

```
        y - half_height,
```

```
        x + half_width,
```

```
        y + half_height )
```

Hier worden de locaties opgeslagen

Maak een lijst met te karteren locaties. Elk item heeft een breedtegraad, lengtegraad en een naam.

We beginnen met slechts twee items voor testen, maar zodra het werkt, breiden we dit uit naar alle locaties waarvoor we afbeeldingen willen maken

```
listofdicstocsv = [  
    {'lat': 50.844441271809465, 'long': -0.2985307978506628, 'officialname': 'Adur  
District Council'},  
    {'lat': 54.71108952940033, 'long': -3.2472347297148736, 'officialname':  
'Allerdale Borough Council'}  
]
```


Hier wordt de **schaal** bepaald

KAARTSCHAAL INSTELLEN: 1 eenheid op de kaart is 175000 in reality

MAP_SCALE = 175000

Hier wordt de uitvoer bepaald

```
# Stel het hulpmiddel in dat de lay-out als afbeeldingsbestand opslaat.  
exporter = QgsLayoutExporter(layout)  
  
settings = QgsLayoutExporter.ImageExportSettings()  
  
# UITVOER AFBEELDINGSGROOTTE INSTELLEN  
  
# 1200x795px voor webgebruik, overeenkomend met de A4-lay-outverhouding in  
# de eerdere code  
  
settings.imageSize = QSize(1200, 795)
```

Loop door elke locatie in de lijst en gebruik de functie om een afbeelding te maken/exporteren

for rec in listofdicstocsv:

 # Lees de breedtegraad, lengtegraad en naam voor deze locatie.

lat = rec.get('lat', float('nan')) # 'nan' gebruikt als vervanging als de waarde ontbreekt

lon = rec.get('long', float('nan'))

officialname = rec.get('officialname', 'unknown')

 # Als de breedtegraad of lengtegraad ontbreekt, sla deze locatie dan over en ga verder.

if math.isnan(lat) or math.isnan(lon):

print(f"skipping '{officialname}': missing lat/long")

continue

 # Verschuif de kaartweergave zodat deze op deze locatie is gecentreerd, zoom in met lengtegraad en breedtegraad

transformed_point = transformer.transform(QgsPointXY(lon, lat))

 # wordt vervolgd...

```
# Extraheer getransformeerde coördinaten

x = transformed_point.x()
y = transformed_point.y()

#Maak een tijdelijke rechthoek rondom het punt om het te centreren
map_item.setExtent( rect_around(x, y, 1000, 1000) )

#Stel een schaal in om het zoomniveau te bepalen. Elke kaart zal dit gebruiken.
map_item.setScale(MAP_SCALE)

#voeg een label toe met die informatie
label.setText( f"{officialname} | scale 1:{MAP_SCALE:,.}" )

#ververs de kaart voor het exporteren
map_item.refresh()

# wordt vervolgd...
```

```
#geef een bestandsnaam - verwijder lastige tekens
safe_name = re.sub( r'^A-Za-z0-9._-]+', '_', officialname).strip('_')

#gebruik dit om het opslagpad te bepalen
out_path = ( f"{out_prefix}" f"{safe_name}.png" )

#en exporteer naar dat pad
result = exporter.exportToImage( out_path, settings )

#als het niet gelukt is
if result != QgsLayoutExporter.Success:

    #toon een foutmelding
    raise RuntimeError(f"Export failed for {out_path}")

#druk een bericht af aan het einde van elke lus
print("wrote", out_path)
```

Hoe doe ik dat
258 keer?

Samenvatting: waar de locaties worden opgeslagen

Maak een lijst met te karteren locaties. Elk item heeft een breedtegraad, lengtegraad en een naam.

We beginnen met slechts twee items voor testen, maar zodra het werkt, breiden we dit uit naar alle locaties waarvoor we afbeeldingen willen maken

```
listofdicstocsv = [  
    {'lat': 50.844441271809465, 'long': -0.2985307978506628, 'officialname': 'Adur  
District Council'},  
    {'lat': 54.71108952940033, 'long': -3.2472347297148736, 'officialname':  
'Allerdale Borough Council'}  
]
```

Een CSV omzetten naar een lijst van dicts

1. **Download de CSV van steden** (klik op de drie puntjes in de rechterbovenhoek)
2. **Typ deze formule in cel J2:**
`=("{ 'lat': "&B2&", 'long': "&C2&", 'officialname': '"&A2&"' }")"`
3. **Kopieer de formule omlaag voor elke rij. Je hebt nu een kolom met dicts.**
4. **Gebruik deze formule om de resultaten samen te voegen & een komma ertussen te zetten**
`=TEXTJOIN(" , ", TRUE, J:J)`
5. **Zoek in je Python-code de openings- en sluitings-[vierkante haken] die je lijst van twee items bevatten, en vervang die twee items door de nieuwe lijst van dicts, met behoud van de vierkante haken.**

{lat: 51.5072, 'long': -0.1275, 'officialname': 'London'}, {lat: 52.48, 'long': -1.9025, 'officialname': 'Birmingham'}, {lat: 50.8058, 'long': -1.0872, 'officialname': 'Portsmouth'}, {lat: 50.9025, 'long': -1.4042, 'officialname': 'Southampton'}, {lat: 52.9561, 'long': -1.1512, 'officialname': 'Nottingham'}, {lat: 51.4536, 'long': -2.5975, 'officialname': 'Bristol'}, {lat: 53.479, 'long': -2.2452, 'officialname': 'Manchester'}, {lat: 53.4094, 'long': -2.9785, 'officialname': 'Liverpool'}, {lat: 52.6344, 'long': -1.1319, 'officialname': 'Leicester'}, {lat: 50.8147, 'long': -0.3714, 'officialname': 'Worthing'}, {lat: 52.4081, 'long': -1.5106, 'officialname': 'Coventry'}, {lat: 54.5964, 'long': -5.93, 'officialname': 'Belfast'}, {lat: 53.8, 'long': -1.75, 'officialname': 'Bradford'}, {lat: 52.9247, 'long': -1.478, 'officialname': 'Derby'}, {lat: 50.3714, 'long': -4.1422, 'officialname': 'Plymouth'}, {lat: 51.4947, 'long': -0.1353, 'officialname': 'Westminster'}, {lat: 52.5833, 'long': -2.1333, 'officialname': 'Wolverhampton'}, {lat: 52.2304, 'long': -0.8938, 'officialname': 'Northampton'}, {lat: 52.6286, 'long': 1.2928, 'officialname': 'Norwich'}, {lat: 51.8783, 'long': -0.4147, 'officialname': 'Luton'}, {lat: 52.413, 'long': -1.778, 'officialname': 'Solihull'}, {lat: 51.544, 'long': -0.1027, 'officialname': 'Islington'}, {lat: 57.15, 'long': -2.11, 'officialname': 'Aberdeen'}, {lat: 51.3727, 'long': -0.1099, 'officialname': 'Croydon'}, {lat: 50.72, 'long': -1.88, 'officialname': 'Bournemouth'}, {lat: 51.58, 'long': 0.49, 'officialname': 'Basildon'}, {lat: 51.272, 'long': -0.529, 'officialname': 'Maidstone'}, {lat: 51.5575, 'long': 0.0858, 'officialname': 'Ilford'}, {lat: 53.39, 'long': -2.59, 'officialname': 'Warrington'}, {lat: 51.75, 'long': -1.25, 'officialname': 'Oxford'}, {lat: 51.5836, 'long': -0.3464, 'officialname': 'Harrow'}, {lat: 52.519, 'long': -1.995, 'officialname': 'West Bromwich'}, {lat: 51.8667, 'long': -2.25, 'officialname': 'Gloucester'}, {lat: 53.96, 'long': -1.08, 'officialname': 'York'}, {lat: 53.8142, 'long': -3.0503, 'officialname': 'Blackpool'}, {lat: 53.4083, 'long': -2.1494, 'officialname': 'Stockport'}, {lat: 53.424, 'long': -2.322, 'officialname': 'Sale'}, {lat: 51.5975, 'long': -0.0681, 'officialname': 'Tottenham'}, {lat: 52.2053, 'long': 0.1192, 'officialname': 'Cambridge'}, {lat: 51.5768, 'long': 0.1801, 'officialname': 'Romford'}, {lat: 51.8917, 'long': 0.903, 'officialname': 'Colchester'}, {lat: 51.6287, 'long': -0.7482, 'officialname': 'High Wycombe'}, {lat: 54.9556, 'long': -1.6, 'officialname': 'Gateshead'}, {lat: 51.5084, 'long': 0.1192, 'officialname': 'Slough'}, {lat: 53.748, 'long': -2.482, 'officialname': 'Blackburn'}, {lat: 51.73, 'long': 0.48, 'officialname': 'Chelmsford'}, {lat: 53.61, 'long': -2.16, 'officialname': 'Rochdale'}, {lat: 53.43, 'long': -1.357, 'officialname': 'Rotherham'}, {lat: 51.584, 'long': -0.021, 'officialname': 'Walthamstow'}, {lat: 51.2667, 'long': -1.0876, 'officialname': 'Basingstoke'}, {lat: 53.483, 'long': -2.2931, 'officialname': 'Salford'}, {lat: 51.4668, 'long': -0.375, 'officialname': 'Hounslow'}, {lat: 51.5528, 'long': -0.2979, 'officialname': 'Wembley'}, {lat: 52.1911, 'long': -2.2206, 'officialname': 'Worcester'}, {lat: 51.4928, 'long': -0.2229, 'officialname': 'Hammersmith'}, {lat: 51.5864, 'long': 0.6049, 'officialname': 'Rayleigh'}, {lat: 51.7526, 'long': -0.4692, 'officialname': 'Hemel Hempstead'}, {lat: 51.38, 'long': -2.36, 'officialname': 'Bath'}, {lat: 51.5127, 'long': -0.4211, 'officialname': 'Hayes'}, {lat: 54.527, 'long': -1.5526, 'officialname': 'Darlington'}, {lat: 50.8352, 'long': -0.1758, 'officialname': 'Hove'}, {lat: 50.85, 'long': 0.57, 'officialname': 'Hastings'}, {lat: 51.655, 'long': -0.3957, 'officialname': 'Watford'}, {lat: 51.9017, 'long': -0.2019, 'officialname': 'Stevenage'}, {lat: 54.69, 'long': -1.21, 'officialname': 'Hartlepool'}, {lat: 53.19, 'long': -2.89, 'officialname': 'Chester'}, {lat: 51.4828, 'long': -0.195, 'officialname': 'Fulham'}, {lat: 52.523, 'long': -1.468, 'officialname': 'Nuneaton'}, {lat: 51.5175, 'long': -0.2988, 'officialname': 'Ealing'}, {lat: 51.8168, 'long': -0.8124, 'officialname': 'Aylesbury'}, {lat: 51.6154, 'long': -0.0708, 'officialname': 'Edmonton'}, {lat: 51.755, 'long': -0.336, 'officialname': 'Saint Albans'}, {lat: 53.789, 'long': -2.248, 'officialname': 'Burnley'}, {lat: 53.7167, 'long': -1.6356, 'officialname': 'Batley'}, {lat: 53.5809, 'long': -0.6502, 'officialname': 'Scunthorpe'}, {lat: 52.508, 'long': -2.089, 'officialname': 'Dudley'}, {lat: 51.4575, 'long': -0.1175, 'officialname': 'Brixton'}, {lat: 51.5111, 'long': -0.3756, 'officialname': 'Southall'}, {lat: 55.8456, 'long': -4.4239, 'officialname': 'Paisley'}, {lat: 51.37, 'long': 0.52, 'officialname': 'Chatham'}, {lat: 53.523, 'long': 0.0554, 'officialname': 'East Ham'}, {lat: 51.346, 'long': -2.977, 'officialname': 'Weston-super-Mare'}, {lat: 54.8947, 'long': -2.9364, 'officialname': 'Carlisle'}, {lat: 54.995, 'long': -1.43, 'officialname': 'South Shields'}, {lat: 55.7644, 'long': -4.1769, 'officialname': 'East Kilbride'}, {lat: 52.8019, 'long': -1.6367, 'officialname': 'Burton upon Trent'}, {lat: 53.9919, 'long': -1.5378, 'officialname': 'Harrogate'}, {lat: 53.099, 'long': -2.44, 'officialname': 'Crewe'}, {lat: 52.48, 'long': 1.75, 'officialname': 'Lewes'}, {lat: 52.37, 'long': -1.26, 'officialname': 'Rugby'}, {lat: 51.623, 'long': 0.009, 'officialname': 'Chingford'}, {lat: 51.5404, 'long': -0.4778, 'officialname': 'Uxbridge'}, {lat: 52.58, 'long': -1.98, 'officialname': 'Walsall'}, {lat: 51.475, 'long': 0.33, 'officialname': 'Grays'}, {lat: 51.3868, 'long': -0.4133, 'officialname': 'Walton upon Thames'}, {lat: 51.4002, 'long': -0.1086, 'officialname': 'Thornthorpe Heath'}, {lat: 51.599, 'long': -0.187, 'officialname': 'Finchley'}, {lat: 51.5, 'long': -0.19, 'officialname': 'Kensington'}, {lat: 52.974, 'long': -0.0214, 'officialname': 'Boston'}, {lat: 50.4353, 'long': -3.5625, 'officialname': 'Paignton'}, {lat: 50.88, 'long': -1.03, 'officialname': 'Waterlooville'}, {lat: 53.875, 'long': -1.706, 'officialname': 'Guisely'}, {lat: 51.5565, 'long': 0.2128, 'officialname': 'Hornchurch'}, {lat: 51.4009, 'long': -0.1517, 'officialname': 'Mitcham'}, {lat: 51.4496, 'long': -0.4089, 'officialname': 'Feltham'}, {lat: 52.4575, 'long': -2.1479, 'officialname': 'Stourbridge'}, {lat: 51.375, 'long': 0.5, 'officialname': 'Rochester'}, {lat: 53.691, 'long': -1.633, 'officialname': 'Dewsbury'}, {lat: 51.5135, 'long': -0.2707, 'officialname': 'Acton'}, {lat: 51.449, 'long': -0.337, 'officialname': 'Twickenham'}, {lat: 53.046, 'long': -2.993, 'officialname': 'Wrexham'}, {lat: 53.279, 'long': -2.897, 'officialname': 'Elesmere Port'}, {lat: 54.66, 'long': -5.67, 'officialname': 'Bangor'}, {lat: 51.019, 'long': -3.1, 'officialname': 'Taunton'}, {lat: 52.7225, 'long': -1.2078, 'officialname': 'Loughborough'}, {lat: 51.54, 'long': 0.08, 'officialname': 'Barking'}, {lat: 51.6185, 'long': -0.2729, 'officialname': 'Edgware'}, {lat: 50.8094, 'long': -0.5409, 'officialname': 'Littlehampton'}, {lat: 51.576, 'long': -0.433, 'officialname': 'Ruislip'}, {lat: 51.4279, 'long': -0.1235, 'officialname': 'Streatham'}, {lat: 51.132, 'long': 0.263, 'officialname': 'Royal Tunbridge Wells'}, {lat: 53.35, 'long': -3.003, 'officialname': 'Bebington'}, {lat: 53.25, 'long': -2.13, 'officialname': 'Macclesfield'}, {lat: 52.3028, 'long': -0.6944, 'officialname': 'Wellingborough'}, {lat: 52.3931, 'long': -0.7229, 'officialname': 'Kettering'}, {lat: 51.878, 'long': 0.55, 'officialname': 'Brentree'}, {lat: 52.2919, 'long': -1.5358, 'officialname': 'Royal Leamington Spa'}, {lat: 54.1108, 'long': -3.2261, 'officialname': 'Barrow in Furness'}, {lat: 56.0719, 'long': -3.4393, 'officialname': 'Dunfermline'}, {lat: 53.3838, 'long': -2.3547, 'officialname': 'Altrincham'}, {lat: 54.0489, 'long': -2.8014, 'officialname': 'Lancaster'}, {lat: 53.4872, 'long': -3.0343, 'officialname': 'Crosby'}, {lat: 53.4457, 'long': -2.9891, 'officialname': 'Bootle'}, {lat: 51.5423, 'long': -0.0026, 'officialname': 'Stratford'}, {lat: 51.0792, 'long': 1.1794, 'officialname': 'Fekstone'}, {lat: 55.945, 'long': -3.994, 'officialname': 'Cumbemauld'}, {lat: 51.208, 'long': -1.48, 'officialname': 'Andover'}, {lat: 51.66, 'long': -3.81, 'officialname': 'Neath'}, {lat: 52.488, 'long': -2.05, 'officialname': 'Rowley Regis'}, {lat: 54.2825, 'long': -0.4, 'officialname': 'Scarborough'}, {lat: 55.98, 'long': -3.17, 'officialname': 'Leith'}, {lat: 50.9452, 'long': -2.637, 'officialname': 'Yeovil'}, {lat: 51.451, 'long': 0.052, 'officialname': 'Eltham'}, {lat: 51.5541, 'long': -0.1744, 'officialname': 'Hampstead'}, {lat: 53.125, 'long': -1.261, 'officialname': 'Sutton in Ashfield'}, {lat: 51.4015, 'long': -0.1949, 'officialname': 'Morden'}, {lat: 51.6444, 'long': -0.1997, 'officialname': 'Barnet'}, {lat: 53.4466, 'long': -2.3086, 'officialname': 'Stretford'}, {lat: 51.408, 'long': -0.022, 'officialname': 'Beckenham'}, {lat: 51.5299, 'long': -0.3488, 'officialname': 'Greenford'}, {lat: 51.702, 'long': -0.035, 'officialname': 'Cheshunt'}, {lat: 53.48, 'long': -2.89, 'officialname': 'Kirkby'}, {lat: 51.07, 'long': -1.79, 'officialname': 'Salsbury'}, {lat: 53.4897, 'long': -2.0952, 'officialname': 'Ashton'}, {lat: 51.394, 'long': -0.307, 'officialname': 'Sutton'}, {lat: 53.716, 'long': -1.356, 'officialname': 'Castleford'}, {lat: 51.4452, 'long': -0.0207, 'officialname': 'Catford'}, {lat: 53.3042, 'long': -1.1244, 'officialname': 'Workshop'}, {lat: 53.7492, 'long': -1.6023, 'officialname': 'Morley'}, {lat: 51.743, 'long': -3.378, 'officialname': 'Merthyr Tydfil'}, {lat: 53.555, 'long': -2.187, 'officialname': 'Middleton'}, {lat: 51.2834, 'long': -0.8456, 'officialname': 'Fleet'}, {lat: 50.85, 'long': -1.18, 'officialname': 'Fareham'}, {lat: 53.4487, 'long': -2.3747, 'officialname': 'Urmston'}, {lat: 51.3656, 'long': -0.1963, 'officialname': 'Sutton'}, {lat: 51.578, 'long': -3.218, 'officialname': 'Caerphilly'}, {lat: 51.128, 'long': -2.993, 'officialname': 'Bridgwater'}, {lat: 51.401, 'long': -1.323, 'officialname': 'Newbury'}, {lat: 51.4594, 'long': 0.1097, 'officialname': 'Welling'}, {lat: 51.46, 'long': -2.505, 'officialname': 'Kingswood'}, {lat: 51.886, 'long': -0.521, 'officialname': 'Dunstable'}, {lat: 51.336, 'long': 1.416, 'officialname': 'Ramsgate'}, {lat: 51.393, 'long': 0.478, 'officialname': 'Strood'}, {lat: 53.5533, 'long': -0.0215, 'officialname': 'Cleethorpes'}, {lat: 51.5932, 'long': -0.3894, 'officialname': 'Pinner'}, {lat: 52.606, 'long': 1.729, 'officialname': 'Great Yarmouth'}, {lat: 52.9711, 'long': -1.3092, 'officialname': 'Ilkeston'}, {lat: 53.653, 'long': -2.632, 'officialname': 'Chorley'}, {lat: 51.37, 'long': 1.13, 'officialname': 'Heme Bay'}, {lat: 51.872, 'long': 0.1725, 'officialname': 'Bishops Cleeve'}, {lat: 53.005, 'long': -1.127, 'officialname': 'Arnold'}, {lat: 52.724, 'long': -1.369, 'officialname': 'Coalville'}, {lat: 51.994, 'long': -0.732, 'officialname': 'Bletchley'}, {lat: 51.9165, 'long': -0.6617, 'officialname': 'Leighton Buzzard'}, {lat: 55.86, 'long': -3.98, 'officialname': 'Ardrie'}, {lat: 55.126, 'long': -1.514, 'officialname': 'Blyth'}, {lat: 51.574, 'long': 0.4181, 'officialname': 'Laindon'}, {lat: 51.684, 'long': -4.163, 'officialname': 'Llanelli'}, {lat: 52.927, 'long': -1.215, 'officialname': 'Beeston'}, {lat: 52.4629, 'long': -1.8542, 'officialname': 'South Heath'}, {lat: 55.0456, 'long': -1.4443, 'officialname': 'Whitley Bay'}, {lat: 53.4554, 'long': -2.112, 'officialname': 'Denton'}, {lat: 52.932, 'long': -1.127, 'officialname': 'West Bridgford'}, {lat: 51.6578, 'long': -0.2722, 'officialname': 'Borehamwood'}, {lat: 56.0011, 'long': -3.7835, 'officialname': 'Falkirk'}, {lat: 53.5239, 'long': -2.3991, 'officialname': 'Walden'}, {lat: 51.5878, 'long': -0.3086, 'officialname': 'Kenton'}, {lat: 54.0819, 'long': -0.1923, 'officialname': 'Bridlington'}, {lat: 54.61, 'long': -1.27, 'officialname': 'Billingham'}, {lat: 52.918, 'long': -0.638, 'officialname': 'Grantham'}, {lat: 55.0097, 'long': -1.4448, 'officialname': 'North Shields'}, {lat: 51.947, 'long': -0.283, 'officialname': 'Hitchin'}, {lat: 52.7858, 'long': -0.1529, 'officialname': 'Spalding'}, {lat: 51.36, 'long': 0.61, 'officialname': 'Rainham'}, {lat: 51.978, 'long': -0.23, 'officialname': 'Letchworth'}, {lat: 51.6114, 'long': 0.5207, 'officialname': 'Wickford'}, {lat: 53.4111, 'long': -2.8403, 'officialname': 'Huyton'}, {lat: 51.6717, 'long': -1.2763, 'officialname': 'Abingdon'}, {lat: 51.32, 'long': -2.208, 'officialname': 'Trowbridge'}, {lat: 52.5812, 'long': -1.093, 'officialname': 'Wigston Magna'}, {lat: 51.606, 'long': -1.241, 'officialname': 'Didcot'}, {lat: 51.433, 'long': -0.933, 'officialname': 'Eaerley'}, {lat: 51.459, 'long': 0.138, 'officialname': 'Bexleyheath'}, {lat: 53.4429, 'long': -1.4698, 'officialname': 'Ecclesfield'}, {lat: 53.698, 'long': -2.461, 'officialname': 'Darwen'}, {lat: 53.5333, 'long': -2.2833, 'officialname': 'Prestwich'}, {lat: 51.602, 'long': -3.342, 'officialname': 'Pontypridd'}, {lat: 55.828, 'long': -4.214, 'officialname': 'Rutherglen'}, {lat: 51.1295, 'long': 1.3089, 'officialname': 'Dover'}, {lat: 50.8365, 'long': -0.7792, 'officialname': 'Chichester'}, {lat: 51.2226, 'long': 1.4006, 'officialname': 'Deal'}, {lat: 51.9, 'long': -1.15, 'officialname': 'Bicester'}, {lat: 51.5467, 'long': -0.37, 'officialname': 'Northolt'}, {lat: 55.7742, 'long': -3.9183, 'officialname': 'Wishaw'}, {lat: 51.3652, 'long': -0.1676, 'officialname': 'Carshalton'}, {lat: 53.001, 'long': -1.197, 'officialname': 'Bulwell'}, {lat: 54.591, 'long': -5.68, 'officialname': 'Newtownards'}, {lat: 54.326, 'long': -2.745, 'officialname': 'Kendal'}, {lat: 55.082, 'long': -1.585, 'officialname': 'Cramlington'}, {lat: 52.3353, 'long': -2.0579, 'officialname': 'Bromsgrove'}, {lat: 51.703, 'long': -3.041, 'officialname': 'Pont-y-pŵll'}, {lat: 51.509, 'long': -0.338, 'officialname': 'Hanwell'}, {lat: 51.2279, 'long': -2.3215, 'officialname': 'Frome'}, {lat: 51.5981, 'long': -0.1149, 'officialname': 'Wood Green'}, {lat: 52.5708, 'long': -2.0457, 'officialname': 'Darlaston'}, {lat: 55.181, 'long': -1.568, 'officialname': 'Ashington'}, {lat: 52.9677, 'long': -2.1327, 'officialname': 'Longton'}, {lat: 52.7661, 'long': -0.886, 'officialname': 'Melton Mowbray'}, {lat: 52.606, 'long': -1.9179, 'officialname': 'Aldridge'}, {lat: 53.5452, 'long': -2.3999, 'officialname': 'Farnworth'}, {lat: 51.552, 'long': -0.097, 'officialname': 'Highbury'}, {lat: 53.3761, 'long': -2.1897, 'officialname': 'Chaddle Hulme'}, {lat: 54.62, 'long': -1.58, 'officialname': 'Newton Aycliffe'}, {lat: 52.4299, 'long': -1.9355, 'officialname': 'Bournville'}, {lat: 52.009, 'long': -0.789, 'officialname': 'Shenley Brook End'}, {lat: 54.85, 'long': -1.83, 'officialname': 'Consett'}, {lat: 51.3211, 'long': -0.1386, 'officialname': 'Coulson'}, {lat: 52.566, 'long': -2.0728, 'officialname': 'Bilston'}, {lat: 52.7001, 'long': -2.5157, 'officialname': 'Wellington'}, {lat: 54.663, 'long': -1.676, 'officialname': 'Bishop Auckland'}, {lat: 52.395, 'long': -1.979, 'officialname': 'Longbridge'}, {lat: 52.614, 'long': -2.004, 'officialname': 'Bloxwich'}, {lat: 51.5557, 'long': 0.2512, 'officialname': 'Uxminster'}, {lat: 53.321, 'long': -3.48, 'officialname': 'Rhyll'}, {lat: 52.267, 'long': -2.153, 'officialname': 'Droitwich'}, {lat: 53.5355, 'long': -2.5658, 'officialname': 'Hindley'}, {lat: 53.549, 'long': -2.529, 'officialname': 'Westhoughton'}, {lat: 51.3589, 'long': 1.4394, 'officialname': 'Broadstairs'}

Gebruik een **slice** in de `for`-lus om te testen:

Door [10:20] achter de lijstnaam te zetten, doorloopt de lus alleen items op de posities 10 t/m 19

```
for rec in listofdicstocsv[10:20]:
```

```
    # Lees de breedtegraad, lengtegraad en naam voor deze locatie.
```

```
    lat = float(rec.get('lat', float('nan'))) # 'nan' wordt gebruikt als vervanging als de waarde ontbreekt
```

```
    lon = float(rec.get('long', float('nan')))
```

```
    officialname = rec.get('officialname', 'unknown')
```

```
    # Als de breedtegraad of lengtegraad ontbreekt, sla deze locatie over en ga verder.
```

```
    if math.isnan(lat) or math.isnan(lon):
```

```
        print(f"skipping '{officialname}': missing lat/long")
```

```
        continue
```

**Hoe doe ik dat
voor gebieden van
verschillende
grootte?**

**We hebben
verschillende schalen
nodig:
Grote conurbatie,
kleiner gebied**

Vibe coding time?

Gebruik meta-prompting om AI eerst de prompt te laten ontwerpen

Voeg het volgende in:

- De context (journalistiek)
- Het doel
- **Gedetailleerde instructies** (specifieke namen, bestanden enz.)

Jij bent mijn mentor, een datajournalist met meer dan tien jaar ervaring in het vak. Je hebt geavanceerde **statistische kennis** én een gezonde **scepsis** bij het omgaan met zowel data als menselijke bronnen.

Ik zal je om hulp vragen bij wat programmeerwerk — je helpt me graag verder, maar **je wilt niet dat ik mijn vaardigheden verlies en te afhankelijk word** van jou, dus je adviezen zijn altijd bedoeld om **mij zelf te laten nadenken, nieuwe vaardigheden en concepten te leren en die te oefenen**.

Wanneer je code aanlevert, **voeg dan commentaar toe dat elke stap uitlegt** alsof je dit uitlegt aan iemand zonder programmeerervaring.

Ik **heb de voorkeur voor eenvoudige code die meerdere regels vereist boven complexe code in één of twee regels**.

Benoem eventuele aannames die je maakt over de data of mijn vraag.

Waarschuw me als de vraag onvoldoende informatie bevat om nauwkeurig te beantwoorden, of als het resultaat misleidend kan zijn zonder aanvullende context.

Bijgevoegd is Python-code die afbeeldingen genereert voor kaarten gecentreerd op honderden locaties. **Pas dit aan zodat het nu die afbeeldingen op twee verschillende schalen genereert.**

Voeg commentaar toe aan de code om het gedeelte te markeren waar dit gebeurt, zodat ik het kan aanpassen en verschillende zoomniveaus kan proberen.

Parameters

[https://github.com/paulbradshaw/QGIS_param/blob/main/qgis_take_pix_t
woSize.py](https://github.com/paulbradshaw/QGIS_param/blob/main/qgis_take_pix_t
woSize.py)

Het gedeelte dat de **schaal**parameters instelt

```
# KAARTSCHALEN INSTELLEN:
```

```
MAP_SCALES = [  
    175000,    # oorspronkelijke schaal  
    120000    # ingezoomde versie  
]
```

Het gedeelte dat die parameters gebruikt

```
# NIEUW: LOOP DOOR MEERDERE SCHALEN

for MAP_SCALE in MAP_SCALES:

    #Maak een tijdelijke rechthoek om het punt heen om het te centreren
    map_item.setExtent(
        rect_around(x, y, 1000, 1000)
    )

    #Pas de huidige schaal toe
    map_item.setScale(MAP_SCALE)

    #voeg een label toe met die informatie
    label.setText(f"{officialname} | scale 1:{MAP_SCALE:,}")
```

...en toevoegen aan de bestandsnaam

#gebruik dat om het opslagpad te bepalen

```
out_path = (
```

```
    f"{out_prefix}"
```

```
    f"{safe_name}_{MAP_SCALE}.png" # NIEUW: VOEG SCHAAL TOE AAN BESTANDSNAAM
```

```
)
```

De kritische vriend: Wat AI je zou moeten vertellen

Aandachtspunten

- Omzetten tussen het CRS van de basiskaart en het shapefile
- Omzetten tussen lay-outgrootte (mm) en imageSize (px)
- Exporteren voor druk/200dpi-resolutie
- Schaalcategorieën in plaats van ruwe getallen (en meer dan twee)

Probeer dit:
Andere optionele
bestanden in de repo

Belangrijkste lessen

- Als je het meer dan eens moet doen...
- Vibe coding is leuk maar *mentor* coding is lonender

Extra bronnen:

- https://github.com/paulbradshaw/QGIS_param
- Tim Green: [Vibe Coding Threatens Journalism: Why Newsrooms Need Governance Now](#)
- [FT newsletter](#) over hun methode waarbij AI helpt bij het programmeren voor het in kaart brengen van overstromingsrisicogebieden